EXP 11

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Introduction to Hyperledger Fabric

Hyperledger Fabric is an open-source, permissioned blockchain platform designed for enterprise use. Unlike public blockchains, it allows only authorized participants to join the network, ensuring privacy, scalability, and security. It follows a modular architecture where components such as consensus, membership services, and smart contracts can be customized.

Fabric Network Architecture

A Hyperledger Fabric network consists of the following components:

- Peers: Nodes that maintain the ledger and execute transactions
- Orderers: Responsible for ordering transactions and creating blocks
- Organizations: Groups of participants in the network
- Channels: Private sub-networks for confidential transactions
- Certificate Authority (CA): Issues digital identities to users

Each participant interacts with the network through secure cryptographic identities.

Prerequitise

1. Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

wsl --list --verbose

If it shows **no distributions**, you must install Ubuntu.

2. Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

wsl --install -d Ubuntu

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

3. Restart Your Computer

After installation finishes, **restart your PC**.
Open Ubuntu: Open Start Menu → Ubuntu
It will ask you to create:

Username

Password

Now Ubuntu is ready.

Implementation

Step 1 — Update the system

code :

```
sudo apt update
sudo apt upgrade -y
```

Explain: apt update refreshes the package index — it tells your system what versions of software are available. apt upgrade installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
snapd.failure.service is a disabled or a static unit not running, not sta
snapd.gpio-chardev-setup.target is a disabled or a static unit not runnir
snapd.snap-repair.service is a disabled or a static unit not running, not
Setting up systemd-sysv (255.4-1ubuntu8.15) ...
Setting up vim (2:9.1.0016-1ubuntu7.10) ...
Setting up libnss-systemd:amd64 (255.4-1ubuntu8.15) ...
Setting up binutils (2.42-4ubuntu2.10) ...
Setting up netplan.io (1.1.2-8ubuntu1~24.04.2) ...
Setting up libpam-systemd:amd64 (255.4-1ubuntu8.15) ...
Setting up cloud-init (25.3-0ubuntu1~24.04.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for rsyslog (8.2312.0-3ubuntu9.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for install-info (7.1-3build2) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
aditya@LAPTOP-VE430QPL:/mnt/c/Windows/System32$
```

Step 2 — Verify prerequisites

code :

sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker --version
docker ps

```
aditya@LAPTOP-VE430QPL: ~
^[[D^[[D^[[D^[[D^[[D^[[D^[[D^[[D^[[D^[[D^[[D^[[Dgroupsaditya@LAPTOP-VE430QPL:~$ groups
aditya adm cdrom sudo dip plugdev users docker
aditya@LAPTOP-VE430QPL:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
aditya@LAPTOP-VE430QPL:~$
```

Explain: Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running docker ps with no output confirms the Docker daemon is running and your user can access it. If docker ps requires sudo, you need to add your user to the docker group.

Step 3 — Clone fabric-samples

code :

git clone https://github.com/hyperledger/fabric-samples

cd fabric-samples

```
aditya@LAPTOP-VE430QPL:~$ git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 10.57 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
aditya@LAPTOP-VE430QPL:~/fabric-samples$
```

Explain: fabric-samples is the official repository containing the test-network scripts, ready-made chaincode examples (including asset-transfer-basic), and configuration files. **Everything else in this guide runs from inside this directory.**

Step 4 — Download the install script and run it

code :

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
aditya@LAPTOP-VE430QPL:~/fabric-samples$
```

Explain: This one script does three things simultaneously. docker pulls all Fabric Docker images — fabric-peer, fabric-orderer, fabric-ccenv, fabric-baseos, fabric-ca, and critically fabric-nodeenv (needed to build Node.js chaincode). binary downloads the CLI tools — peer, configtxgen, configtxlator — into ~/fabric-samples/bin. samples ensures the chaincode sample directories are present. Without fabric-nodeenv, chaincode deployment fails later.

Step 5 — Verify Docker images are present

code :

docker images | grep fabric

```
aditya@LAPTOP-VE430QPL:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --f
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-nodeenv:2.5              17e2d447ca0d        570MB          146MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
```

Explain: Confirm hyperledger/fabric-nodeenv:2.5 is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, deployCC will fail with a "No such image" error.

Step 6 — Set PATH and config path

code :

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

```
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
aditya@LAPTOP-VE430QPL:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
```

Explain: The peer binary lives in ~/fabric-samples/bin, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". FABRIC_CFG_PATH tells the peer CLI where to find core.yaml, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

code :

peer version

You should see Version: v2.5.15.

Step 7 — Start the network and create the channel

code :

cd ~/fabric-samples/test-network
./network.sh up createChannel

```
++ configtxlator compute_update --channel_id mychannel --original /home/aditya/fabric
acts/original_config.pb --updated /home/aditya/fabric-samples/test-network/channel-ar
 /home/aditya/fabric-samples/test-network/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /home/aditya/fabric-samples/test-network/channe
 common.ConfigUpdate --output /home/aditya/fabric-samples/test-network/channel-artifa
++ jq .
+++ cat /home/aditya/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}}
el_id":' '"mychannel",' '"isolated_data":' '{},' '"read_set":' '{' '"groups":' '{' '"
'"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Admins":
y":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"polic
'"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Wr
'"policy":' null, '"version":' '"0"' '}' '},' '"values":' '{' '"MSP":' '{' '"mod_poli
ion":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":'
:' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"versi
 '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{
olicies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '
d_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_po
ersion":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"versi
' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '
le.com",' '"port":' 9051 '}' ']' '},' '"version":' '"0"' '},' '"MSP":' '{' '"mod_poli
ion":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",' '"policies":'
:' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"versi
++ configtxlator proto_encode --input /home/aditya/fabric-samples/test-network/channe
be.json --type common.Envelope --output /home/aditya/fabric-samples/test-network/chan
2026-04-12 10:33:51.911 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and ord
2026-04-12 10:33:51.930 UTC 0002 INFO [channelCmd] update -> Successfully submitted c
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ S
```

Explain: This single command does everything needed to have a running network with a channel. Internally it uses cryptogen to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — orderer.example.com on port 7050, peer0.org1.example.com on port 7051, and peer0.org2.example.com on port 9051. It then runs configtxgen to create mychannel.block (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is mychannel.

Step 8 — Deploy the chaincode

code :

*./network.sh deployCC *
*-ccn basic *
*-ccp ../asset-transfer-basic/chaincode-javascript *
-ccl javascript

```
tatus:200 payload:"\nJbasic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629\022\tbasic_1.0" >
2026-04-12 10:35:03.724 UTC 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identif
er: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629
Chaincode is installed on peer0.org2
++ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
++ sed -n '/Version:/{s/.*Sequence: //; s/, Endorsement Plugin:.*$//; p;}'
Error: query failed with status: 404 - namespace basic is not defined
+ COMMITTED_CC_SEQUENCE=
+ res=0
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629$'
+ res=0
basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629
Query installed successful on peer0.org1 on channel
Using organization 1
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --c
file /home/aditya/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-ce
t.pem --channelID mychannel --name basic --version 1.0 --package-id basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b
4c035bee15cbcc7f59b629 --sequence 1
+ res=0
2026-04-12 10:35:06.074 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b4a386ee79f9d1bb27a656888da1c7abbe0e3478233877
5a59762bb78b7483d] committed with status (VALID) at localhost:7051
Chaincode definition approved on peer0.org1 on channel 'mychannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...
```

Explain: -ccn basic is the name your chaincode will be called by in all future commands. -ccp is the path to the chaincode source code. -ccl javascript tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a .tar.gz, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (dev-peer0.org1... and dev-peer0.org2...).

Step 9 — Set peer identity environment variables

code :

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.
ers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.exam
Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$
```

Explain: The peer CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. CORE_PEER_LOCALMSPID declares "I am Org1". CORE_PEER_MSPCONFIGPATH points to the Admin user's private signing key and certificate. CORE_PEER_TLS_ROOTCERT_FILE provides the CA certificate to verify the peer's TLS identity. CORE_PEER_ADDRESS is which peer to send requests to. These must be re-exported in every new terminal session.

Step 10 — Initialize the ledger with sample data

code :

*peer chaincode invoke *
*-o localhost:7050 *
*--ordererTLSHostnameOverride orderer.example.com *
*--tls *
*--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem *
*-C mychannel *
*-n basic *
*--peerAddresses localhost:7051 *
*--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt *
*--peerAddresses localhost:9051 *
*--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt *
*--waitForEvent *

-c '{"function":"InitLedger","Args":[]}'

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"InitLedger","Args":[]}'
2026-04-12 10:35:54.917 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d1b7b843a503299e5584c60f139e5828643fb8221008a5c
7247dc5e724cdebd5] committed with status (VALID) at localhost:9051
2026-04-12 10:35:54.917 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d1b7b843a503299e5584c60f139e5828643fb8221008a5c
7247dc5e724cdebd5] committed with status (VALID) at localhost:7051
2026-04-12 10:35:54.917 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: stat
us:200
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$
```

Explain: invoke is used for any operation that writes to the ledger. -o localhost:7050 is the orderer address. --tls and --cafile enable TLS and provide the CA cert to verify the orderer. -C mychannel and -n basic identify which channel and chaincode. The two --peerAddresses and --tlsRootCertFiles pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in ENDORSEMENT_POLICY_FAILURE. --waitForEvent blocks until both peers confirm the transaction is committed (status VALID) rather than just submitted. InitLedger seeds the ledger with six sample assets.

Step 11 — Query all assets

code :

*peer chaincode query *
*-C mychannel *
*-n basic *
-c '{"Args":["GetAllAssets"]}'

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"Appra
Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"gre
","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","O
e":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15
t"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$
```

Explain: query is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by InitLedger:

```
json
[
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800}
]
```

Step 12 — Read a specific asset

code :

```
peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["ReadAsset","asset5"]}'
```

Result:

```
json
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}
```

Explain: ReadAsset takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

Step 13 — Create a new asset

code :

```
peer chaincode invoke \
```

```
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-
network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
```

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode inv
oke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
\
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/pee
rOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
\
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","
1000"]}'
[[D2026-04-12 10:38:14.780 UTC 0001 INFO [chaincodeCmd] ClientWait -> t
xid [6649720b932a69f998964a84d80c725ed363ad9d870e5264e36814886248b740] c
ommitted with status (VALID) at localhost:7051
2026-04-12 10:38:14.780 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid
[6649720b932a69f998964a84d80c725ed363ad9d870e5264e36814886248b740] commi
tted with status (VALID) at localhost:9051
2026-04-12 10:38:14.781 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":
\"asset10\",\"Color\":\"blue\",\"Size\":10,\"Owner\":\"Pranav\",\"Apprai
sedValue\":1000}"
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$
```

Explain: CreateAsset takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

code :

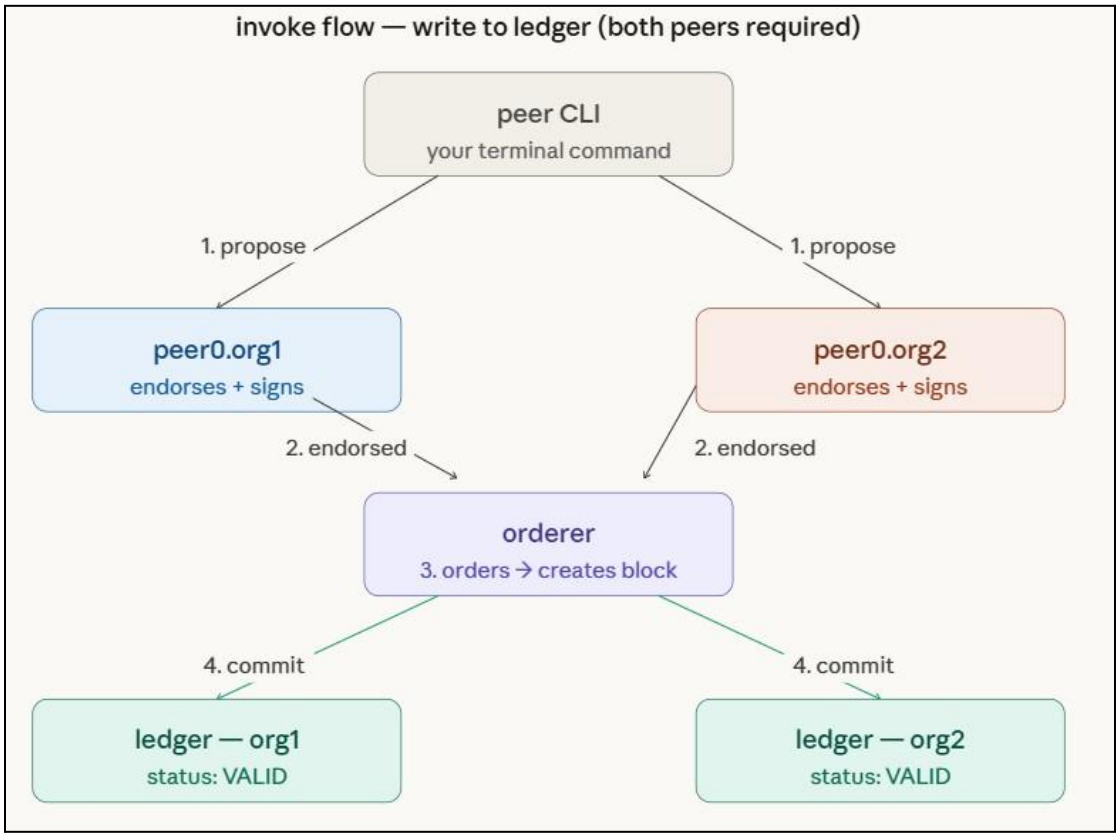
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args"
","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$
```

{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}

The transaction flow showing what happens inside Fabric every time you run invoke or query

Shows what happens inside Fabric every time you run peer chaincode invoke — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.



Step 14 — Transfer asset ownership

code :

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-
network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
```

*--peerAddresses localhost:9051 *
--tlsRootCertFiles
*$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt *
*--waitForEvent *
-c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.co
/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org
.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org
.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Rahul"]}'
2026-04-12 10:39:13.111 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d11b472d5c82fbbe50b484ce283d8ccdd2302c005b424c
af2095b19be35ebe9] committed with status (VALID) at localhost:7051
2026-04-12 10:39:13.111 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d11b472d5c82fbbe50b484ce283d8ccdd2302c005b424c
af2095b19be35ebe9] committed with status (VALID) at localhost:9051
2026-04-12 10:39:13.112 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: sta
us:200 payload:"Pranav"
```

Explain: TransferAsset takes [assetID, newOwner]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("Pranav") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

Verify the ownership change:

code :
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset
","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

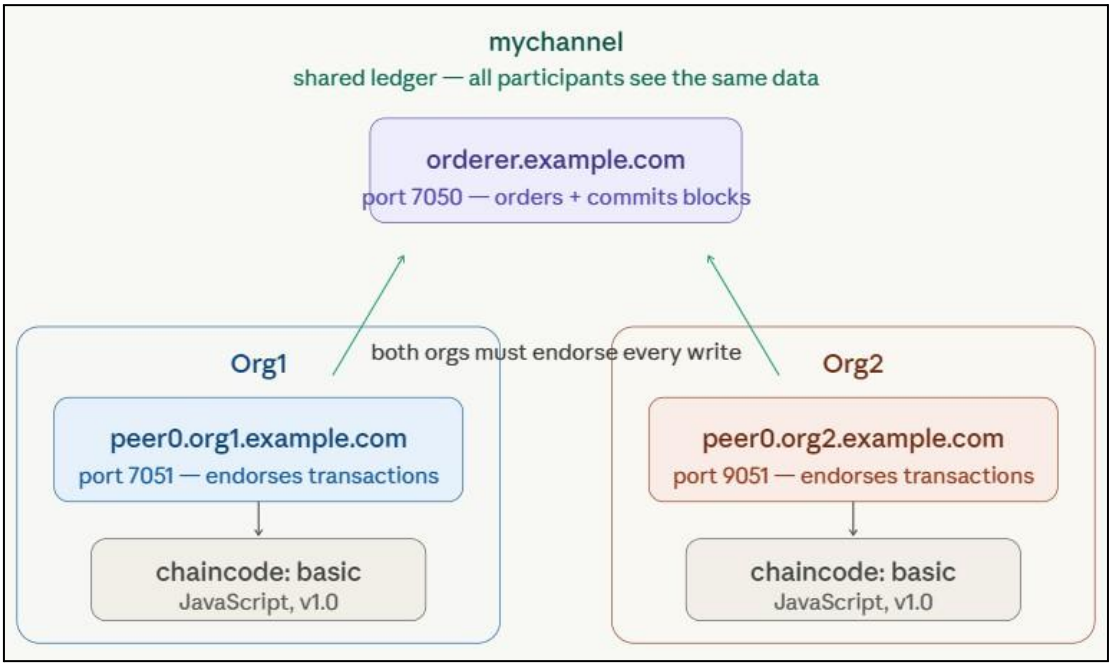
Owner is now **Rahul**

Step 15 — To stop/bring down the Hyperledger Fabric network, run:

./network.sh down

```
aditya@LAPTOP-VE430QPL:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
ERROR: Top level object in './addOrg3/compose/docker/docker-compose-couch-org3.yaml' needs to be an object not '<class '
NoneType'>'.
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
2f3f6b17b342
ef9bf7358560
7f43f96760cc
f3ea09f3c9ef
78a21e798e4b
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc
2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:867acf907faeeef18f7279aa48d91fe77e9b28989fd344ab9534b4b1cd1a4c19
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f
4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:c611123e1301ccee53a22387367ec988efd36cd9a979aa8330c6e054aa3ea168
```

Network architecture showing all components and how they relate



Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.